

From Exhausted to Empowered: Experiments, Data Analysis, and Health Estimation of Second-life Batteries

Code Guide

This guide is created to aid users in understanding and running the code provided alongside the work done in the paper “From Exhausted to Empowered: Experiments, Data Analysis, and Health Estimation of Second-life Batteries”. The outline contains different parts of the codebase and dataset that can help the users understand better the structure of the code and use it with ease. It is recommended to use at least MATLAB 2023a when using these scripts directly.

Outline:

1. Dataset structure
 - a. Raw data (excel)
2. Processed data
 - a. Raw data (.mat)
 - b. Featurized data (.mat)
3. Custom classes
 - a. HRAW.m
 - b. HSUM.m
 - c. HANA.m
 - d. HPLOT.m
 - e. HML.m
4. Code execution
 - a. Data processing
 - b. Machine learning models
 - c. Adaptive ML model
5. Contact

1. Dataset structure:

Experiments and data collection for this work is done by Relyion Energy Inc. based in Santa Clara, CA. For any queries related to the experiments and data collection, please contact Surinder Singh (surinder.singh@relyionenergy.com). The total size of the dataset is approximately **15GB**. Data for this work is obtained from two different sources: battery cyclers and thermocouple sensors. Raw data measured by the thermocouples is not directly integrated into the raw data measured by the battery cyclers. Therefore, data from these two sources are combined during post-processing.

a. Raw data (excel)

The raw data provided with this work is in the form of Excel files (.xlsx and .csv). Raw excel data is split into the following folders:

- **Aging**: Contains folders for each cell with files for the aging cycles data of the experimental campaign. Details of the aging protocol are given in the main paper in Section 2.1 and Figure 1.
- **Cby20**: Contains folders for each cell with files containing the C/20 capacity test data. Details of C/20 test are given in Supplemental Information (SI) Section 2 and main paper Figure 1.
- **HPPC**: Contains folders for each cell with files containing the HPPC test data. Details of HPPC test are given in SI Section 2 and main paper Figure 1.
- **OCV**: Contains folders for each cell with files containing the C/40 OCV test data. Details of C/40 OCV test are given in SI Section 2 and main paper Figure 1.
- **Temperature Data**: Contains .csv files of raw temperature data for all cells. Details of temperature data are given in main paper Section 2.1 and SI Section 2. The thermocouple IDs (two per cell) below correspond to the different cells in the temperature data files.

Cell name	Thermocouple ID	Cell name	Thermocouple ID
Cell 1.1	e5ae	Cell 2.1	6e61
	762d		b102
Cell 1.2	b447	Cell 2.2	3309
	ac87		2858
Cell 1.3	a710	Cell 2.3	29ab
	e4fd		366e
Cell 1.4	67bd	Cell 2.4	410b
	ba9d		959e

2. Processed data

a. Raw data (.mat)

To make excel raw files easy to use in MATLAB, dedicated classes are used to process and convert the files into .mat raw files (details about classes in Section 3). The total size of the .mat raw data is approximately **13.6GB**. Processed raw data is split into the following folders:

- **aging_cycle_raw:** Contains folder for each cell in which .mat files contain all the aging raw data. Due to the size of the aging data, 10 .mat files are used to store the raw aging data for each cell. The files are named as cell_{num}_aging_cycle_raw_{file number}.mat where {num} = {1p1,1p2,...,2p4} and {file number} = {1,2,3,...,10}.
- **rpt_cycle_raw:** Contains only .mat files (one for each cell) that contains the raw RPT data (C/20, HPPC, OCV). Each file contains a column "rpt_type" that can be used to access the data of a specific RPT test. File naming convention is cell_{num}_rpt_cycle_raw.mat where {num} = {1p1,1p2,...,2p4}.
- **temperature_raw:** Contains only .mat files that contain the raw temperature data separated by cell number and thermocouple ID. File naming convention is sensor_{ID}_temperature_raw.mat where {ID} = {1p1a,1p1b,...,2p4a,2p4b} and a,b are one of the two thermocouples. For example, file sensor_1p1a_temperature_raw.mat contains the temperature data recorded by thermocouple with ID: e5ae.

This conversion from excel to .mat files allows faster access of the data without the need to work with the raw excel data repeatedly.

b. Featurized data (.mat)

Featurization of data entails extracting features from different sources of data e.g., aging, RPT, and temperature. These features are useful to understand which part of the data is more important than others and this information can be used to build efficient models. For more details, see SI Section 3.2. Data featurization also combines different sources for data together to have a coherent data structure for analysis and model building. Featurized data is split into the following folders:

- **aging_cycle_summary:** Contains .mat files for each cell containing various features extracted from the aging data. For example, $Q_{ch,aging}$ is the Ah-throughput from the charging portion of the aging cycle profile and this is available for every single cycle in the dataset.
- **aging_cycle_summary_clean:** Same as the data in aging_cycle_summary but with two additional features avg_temperature_degC_ and aging_ah_accumulate_ah_ which are the average temperature and accumulated Ah-throughput for each cycle, respectively. Data in this folder is also cleaned which includes removing NaNs wherever possible without losing information from other features.
- **rpt_cycle_summary:** Contains .mat files for each cell with features from each RPT test. Temperature and Ah-throughput data is also included as features.

Same as rpt_cycle_raw, the column “rpt_type” can be used to access features for a specific RPT test.

- **temperature_clean**: Contains only .mat files of temperature data for each cell which is filtered and cleaned to remove noisy data and missing data. Temperature data in this folder is generated by combining temperature measurements from both thermocouples. If data is available from both, then it is averaged; otherwise, data from the available thermocouple is used.

Note: There are cases where temperature data is not measured by both sensors which results in missing temperature data.

Figure below shows the folders mentioned above. Users can also refer to SI Figure S4 to understand how data processing works.

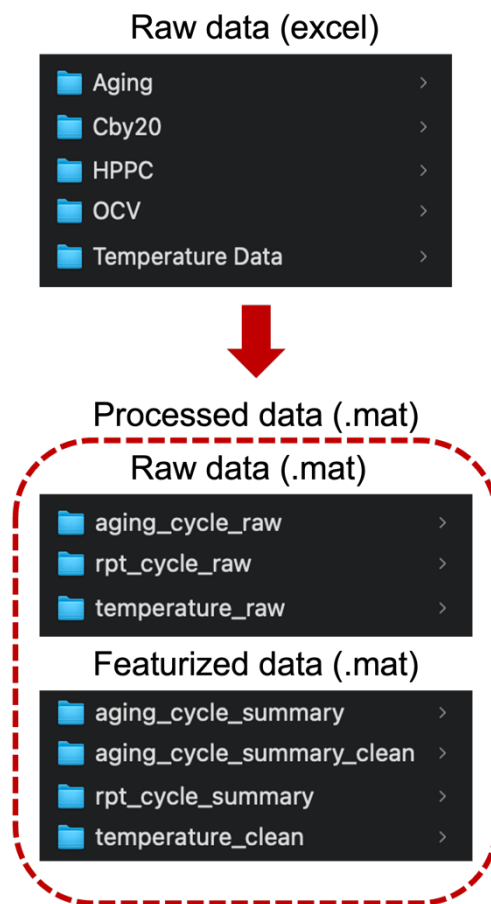


Figure 1: Dataset structure showing folders in the raw excel data, raw .mat data, and featurized data

3. Custom classes:

Elements of Object-Oriented Programming (OOP) have been incorporated in the code to make it modular, readable, and easy-to-debug. Custom classes are designed that are intended to address different aspects of this work. For example, some classes are used in data processing while other classes are used for model development. These classes are defined as follows:

a. HRAW.m: Processes raw excel data to generate raw .mat data. This class contains the following functions:

- i. generate_rpt_raw()
- ii. generate_aging_raw()
- iii. generate_temperature_raw()
- iv. split_temperature_raw()
- v. generate_cover20_raw()
- vi. generate_ocv_raw()
- vii. generate_hppc_raw()

b. HSUM.m: Processes raw .mat data to generate featurized .mat data. These functions perform featurization, filtering, cleaning, and concatenation of data to make it available in a compiled format for each cell.

- i. generate_aging_summary()
- ii. generate_rpt_summary()
- iii. update_summary_clean_by_temperature()
- iv. update_summary_clean_by_ah_cycle()
- v. clean_temperature()
- vi. generate_hppc_summary()
- vii. get_aging_capacity()
- viii. get_aging_resistance()
- ix. get_aging_ocv()
- x. generate_cover20_summary()
- xi. generate_ocv_summary()
- xii. clean_aging()
- xiii. temperature_sensor_fusion()
- xiv. get_resistance_hppc()
- xv. calculate_resistance_mean()
- xvi. calculate_voltage_mean()
- xvii. average_temp()

c. HANA.m: Contains functions that are used to perform different analysis e.g. studying the effect of temperature on capacity.

- i. pred_res_diag_scatter()
- ii. pred_res_indi_cell()
- iii. pred_res_indi_cell_allin1()
- iv. pred_res_indi_cell_errorhist()
- v. data_preprocess_cover20_capacity{x}() where x=2,3,5,6,7,8
- vi. data_preprocess_cover20_capacity(x)_nom() where x=3,4
- vii. data_preprocess_refine()
- viii. traj_dist()

- ix. complete_dist_analysis()
- x. endpoint_dist_analysis()
- xi. tempEffectAnalysis2()
- xii. removeNansAndInterpolate()
- xiii. calculateTempEffect()
- xiv. calculateTempEffect2()

d. HPLLOT.m: Contains functions that are used to generate some of the figures in the paper.

- i. plot_aging_cycle_sum_1()
- ii. test_func_measure()
- iii. plot_aging_cycle_ah_temp_time2()
- iv. plot_aging_cycle_ah_c_over_20_ah()
- v. plot_aging_cycle_ah_c_over_20_ah_delete_first()
- vi. plot_aging_cycle_ah_c_over_40_ah()
- vii. plot_aging_res_ah_rpt_res_ah()
- viii. plot_aging_res_ah_neg_c_over_20_ah()
- ix. plot_aging_cycle_ah_rpt_ah_filtering()
- x. plot_aging_temp_time_rpt_time()
- xi. plot_aging_temp_time_rpt_time_filtering()
- xii. plot_aging_cycle_ah_temp_ah()
- xiii. plot_aging_cycle_cycle_temp_cycle()
- xiv. plot_capacity_ah()
- xv. plot_c_ah_for_paper()
- xvi. plot_c_ah_time_for_python()
- xvii. plot_temp_ah()
- xviii. plot_temp_ah_for_paper()
- xix. coplot_temp_ah_for_paper()
- xx. plot_hppc_resistance()
- xxi. plot_grid_ah_throughput()

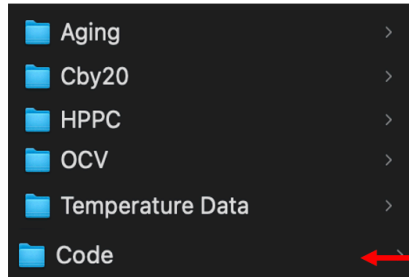
e. HML.m: Contains functions that are used to develop and train both online and offline data-driven models shown in the paper.

- i. train_enr()
- ii. train_clustering()
- iii. train_weighted_enr()
- iv. train_svm()
- v. train_rfr()
- vi. train_gpr()
- vii. show_error()
- viii. id_cluster_key()
- ix. predict_ada_static()
- x. predict_ada_dynamic()

4. Code execution:

Before running the code, ensure that all the code files are present inside a single folder and this folder should be in the same directory as all the other raw data (excel) folders. An example of this is given below.

Raw data (excel)



Contains all the MATLAB scripts

a. Data processing

For data processing, all the code is present in the main.m file which is used to initiate class handlers and run functions to process code from raw excel all the way to featurized data. The following steps highlight running different parts of the script:

1. All the functions inside different classes are accessed through class handlers that are initiated in the main.m file. A snippet is shown below.

```
%% Intialize Pre-load parameters
clc; close all; clear all;
addpath(pwd);
hraw1 = HRAW; hsum1 = HSUM; hplot1 = HPL0T;
hana1 = HANA; hml1 = HML; htd1 = HTEMPDECOUPLE;
```

2. Once class handlers are initiated, different functions can be accessed through them. Here, we show how to go from raw data (excel) -> raw data (.mat) -> featurized data (.mat). After running the section for class handlers, run the section below to generate the raw (.mat) temperature data separated for each cell.

```
%% Generate raw temperature data
clc;
tic; hraw1.generate_temperature_raw();
hraw1.split_temperature_raw(); toc; % split the temperature into each sensors
```

3. Similarly, to get the raw/summary aging and RPT data, the following sections should be executed.

```
%% Generate aging raw and summary aging data
clc;
tic; hraw1.generate_aging_raw(["1p1","1p2","1p3","1p4","2p1","2p2","2p3","2p4"]); toc;
tic; hsum1.generate_aging_summary(["1p1","1p2","1p3","1p4","2p1","2p2","2p3","2p4"]); toc;

%% Generate RPT raw and summary rpt data
clc;
tic; hraw1.generate_rpt_raw(["1p1","1p2","1p3","1p4","2p1","2p2","2p3","2p4"]); toc;
tic; hsum1.generate_rpt_summary(["1p1","1p2","1p3","1p4","2p1","2p2","2p3","2p4"]); toc;
```

4. In the code snippet above, this is done for all the cells; however, it can also be done for a specific cell by only including that cell in the function argument.

Disclaimer: processing raw aging data for all the cells can take **many hours (close to a day)** due to the amount of data. It is recommended that users try the data processing part of the code with one or two cells to understand the working of the code.

These sections generate the raw data. The “clean” and “update” sections process the raw temperature data further and then combine it with the aging and RPT data according to the time steps. The main.m script gives comments to help the user with this execution.

Note: All the raw and summary folders will be generated in the **Code** folder.

b. Machine learning models

Once all the required data processing is completed through main.m, machine learning (ML) models can be trained using main_ML.m script. This script allows users to set training/test sets, perform feature selection, and train four ML models that are given in the paper. The shared version of the script generates results given in the paper; users can change various parts of the script to get other results. Some main aspects of the script are given below:

5. For different test sets, the feature selection (mRMR) gives different features that can give the best results. This can be adjust using the line below and it would select the top x features from the histogram generated. The number of features shown is what is used in the paper.

```
ResTab.end_feature = [3, 7, 3, 6]'; % this gives you freedom to select the top x features
```

6. This loop is intended to choose different test sets. The index w can select different cells that can be used as the test set. For example, in the code snippet below, w=4 selects Cell 1.4 and Cell 2.4 as the test cells while the other 6 cells are used for training.

```
for w = 4:4
    close all;
    index_set = ["1p1","1p2","1p3","1p4","2p1","2p2","2p3","2p4"];
    %test_set = ["1p4", "2p4"]; ["1p4", "2p1"]
    test_set = [ResTab.test_set_num1(w), ResTab.test_set_num2(w)];
```

7. Using the code below, different models can be trained separately or even simultaneously. **Note:** ENR is the fastest during training while other models can take a longer time to finish training. Our suggestion is to save the trained model for repeated use.

<pre> %% Compare different ML methods rng(1,'twister') % fixed the random seed, important % Method 0 - Elastic Net Regression tic; Mdl = hml1.train_enr(D_train, Y_train); save('Mdl_enr.mat', 'Mdl'); toc;% ENR Regression % Method 1 - %Support Vector Regression tic; Mdl = hml1.train_svm(D_train, Y_train); save('Mdl_svm.mat', 'Mdl'); toc;% SVM Regression % Method 2 - Random Forest Regression tic; Mdl = hml1.train_rfr(D_train, Y_train); save('Mdl_rfr.mat', 'Mdl'); toc;% Random Forest Regression % Method 3 - Gaussian Process Regression tic; Mdl = hml1.train_gpr(D_train, Y_train); save('Mdl_gpr.mat', 'Mdl'); toc;% GPR Regression % Method 4 - Dummy Prediction Using Clustering tic; Mdl = hml1.train_clustering(D_train, Y_train); toc; % Dummy Prediction </pre>	
--	--

All the results including errors are saved in a structure called ResTab and it can be used to access everything.

c. Adaptive ML model

The script for adaptive ML results are similar to main_ML.m and it is called main_ML_adaptive.m. This script includes additional functions for the HML class that are used to train and run the adaptive ML model. The code snippet below shows the part of the code that trains the clustering-based model.

<pre> %% Train different ML methods clc; rng(1,'twister') % fixed the random seed, important % Method 0 - Elastic Net Regression tic; Mdl = hml1.train_enr(D_train, Y_train); save("Mdl_enr.mat", "Mdl"); toc;% ENR Regression % Method 1 - Support Vector Regression tic; Mdlsvm = hml1.train_svm(D_train, Y_train); toc;% SVM Regression % Method 2 - Random Forest Regression tic; Mdlrfr = hml1.train_rfr(D_train, Y_train); toc;% Random Forest Regression % Method 3 - Gaussian Process Regression tic; Mdlgpr = hml1.train_gpr(D_train, Y_train); toc;% GPR Regression % Method 4 - Dummy Prediction Using Clustering tic; Mdl = hml1.train_clustering(X_train, Y_train, 8); save("Mdlclu.mat", "Mdl"); toc; % Dummy Prediction </pre>	
---	--

5. Contact

For any further queries, contact the authors Xiaofan Cui (cuixf@seas.ucla.edu) and Muhammad Aadil Khan (maadilk@stanford.edu).